

EXERCISE 4 - parallel for vs parallel for collapse(2)

Files : exercise04.c (original code)

exercise04_collapse.c (with collapse(2))

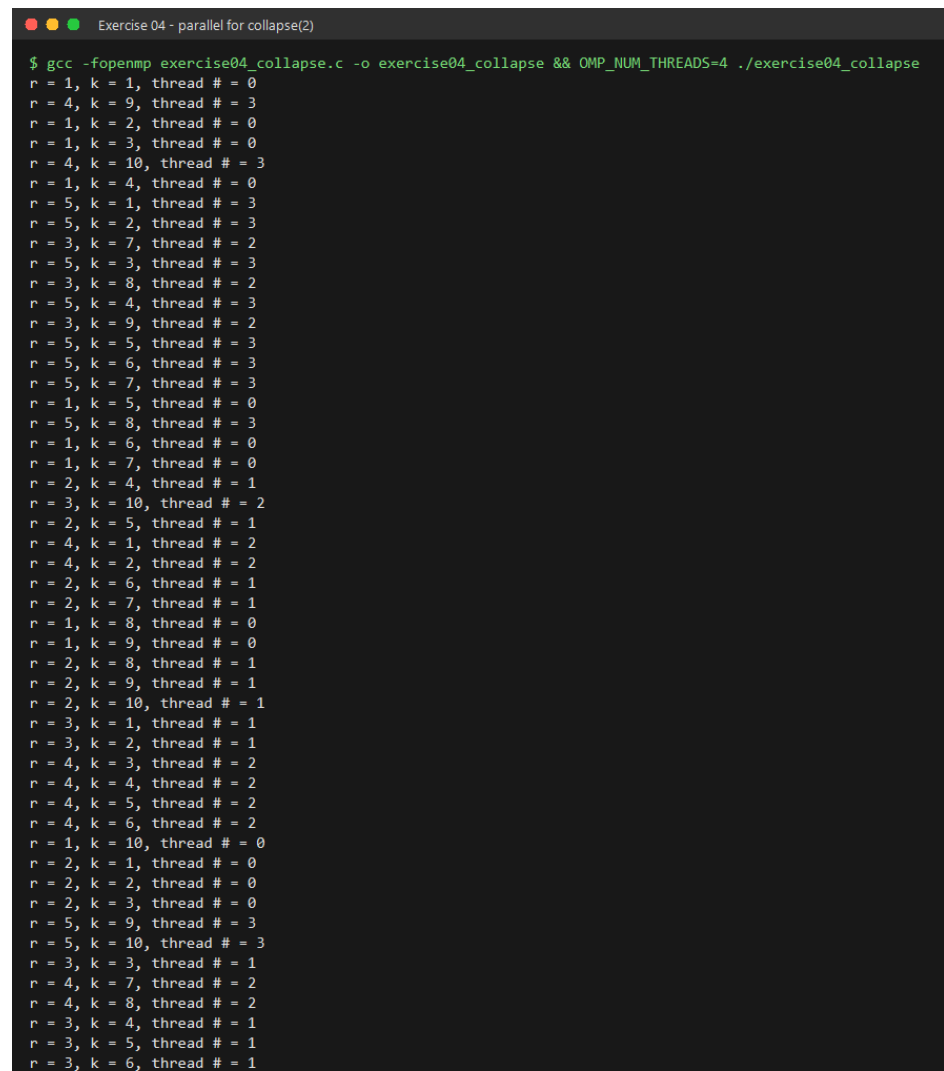
Compile / run

```
gcc -fopenmp exercise04.c -o exercise04
```

```
OMP_NUM_THREADS=4 ./exercise04
```

```
gcc -fopenmp exercise04_collapse.c -o exercise04_collapse
```

```
OMP_NUM_THREADS=4 ./exercise04_collapse
```



```
Exercise 04 - parallel for collapse(2)

$ gcc -fopenmp exercise04_collapse.c -o exercise04_collapse && OMP_NUM_THREADS=4 ./exercise04_collapse
r = 1, k = 1, thread # = 0
r = 4, k = 9, thread # = 3
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 4, k = 10, thread # = 3
r = 1, k = 4, thread # = 0
r = 5, k = 1, thread # = 3
r = 5, k = 2, thread # = 3
r = 3, k = 7, thread # = 2
r = 5, k = 3, thread # = 3
r = 3, k = 8, thread # = 2
r = 5, k = 4, thread # = 3
r = 3, k = 9, thread # = 2
r = 5, k = 5, thread # = 3
r = 5, k = 6, thread # = 3
r = 5, k = 7, thread # = 3
r = 1, k = 5, thread # = 0
r = 5, k = 8, thread # = 3
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 2, k = 4, thread # = 1
r = 3, k = 10, thread # = 2
r = 2, k = 5, thread # = 1
r = 4, k = 1, thread # = 2
r = 4, k = 2, thread # = 2
r = 2, k = 6, thread # = 1
r = 2, k = 7, thread # = 1
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 2, k = 8, thread # = 1
r = 2, k = 9, thread # = 1
r = 2, k = 10, thread # = 1
r = 3, k = 1, thread # = 1
r = 3, k = 2, thread # = 1
r = 4, k = 3, thread # = 2
r = 4, k = 4, thread # = 2
r = 4, k = 5, thread # = 2
r = 4, k = 6, thread # = 2
r = 1, k = 10, thread # = 0
r = 2, k = 1, thread # = 0
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 5, k = 9, thread # = 3
r = 5, k = 10, thread # = 3
r = 3, k = 3, thread # = 1
r = 4, k = 7, thread # = 2
r = 4, k = 8, thread # = 2
r = 3, k = 4, thread # = 1
r = 3, k = 5, thread # = 1
r = 3, k = 6, thread # = 1
```

1. #pragma omp parallel for

Only the OUTER loop ($r = 1..5$) is parallelised. The 5 iterations of r are split between the 4 threads; each thread then runs the WHOLE inner loop ($k = 1..10$) for its own r values.

thread	r values	iterations
--------	------------	------------

-----	-----	-----
-------	-------	-------

0	$r = 1, r = 2$	20
---	----------------	----

1	$r = 3$	10
---	---------	----

2	$r = 4$	10
---	---------	----

3	$r = 5$	10
---	---------	----

5 does not divide by 4, so the work is UNBALANCED - thread 0 does twice as much as the others.

2. #pragma omp parallel for collapse(2)

collapse(2) merges the two loops into ONE iteration space of $5 \times 10 = 50$ (r,k) pairs and distributes those between the threads.

thread	(r,k) pairs	iterations
--------	---------------	------------

-----	-----	-----
-------	-------	-------

0	$(1,1) \dots (2,3)$	13
---	---------------------	----

1	$(2,4) \dots (3,6)$	13
---	---------------------	----

2	$(3,7) \dots (4,8)$	12
---	---------------------	----

3 (4,9) ... (5,10) 12

The split no longer follows the r boundaries (thread 0 and 1 share r = 2), and the load is now BALANCED (13/13/12/12).

```
Exercise 04 - parallel for

$ gcc -fopenmp exercise04.c -o exercise04 && OMP_NUM_THREADS=4 ./exercise04
r = 3, k = 1, thread # = 1
r = 5, k = 1, thread # = 3
r = 3, k = 2, thread # = 1
r = 1, k = 1, thread # = 0
r = 3, k = 3, thread # = 1
r = 3, k = 4, thread # = 1
r = 5, k = 2, thread # = 3
r = 5, k = 3, thread # = 3
r = 4, k = 1, thread # = 2
r = 5, k = 4, thread # = 3
r = 4, k = 2, thread # = 2
r = 5, k = 5, thread # = 3
r = 5, k = 6, thread # = 3
r = 1, k = 2, thread # = 0
r = 4, k = 3, thread # = 2
r = 4, k = 4, thread # = 2
r = 4, k = 5, thread # = 2
r = 1, k = 3, thread # = 0
r = 3, k = 5, thread # = 1
r = 5, k = 7, thread # = 3
r = 3, k = 6, thread # = 1
r = 5, k = 8, thread # = 3
r = 3, k = 7, thread # = 1
r = 3, k = 8, thread # = 1
r = 3, k = 9, thread # = 1
r = 3, k = 10, thread # = 1
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 4, k = 6, thread # = 2
r = 4, k = 7, thread # = 2
r = 4, k = 8, thread # = 2
r = 4, k = 9, thread # = 2
r = 5, k = 9, thread # = 3
r = 5, k = 10, thread # = 3
r = 4, k = 10, thread # = 2
r = 2, k = 1, thread # = 0
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 2, k = 4, thread # = 0
r = 2, k = 5, thread # = 0
r = 2, k = 6, thread # = 0
r = 2, k = 7, thread # = 0
r = 2, k = 8, thread # = 0
r = 2, k = 9, thread # = 0
r = 2, k = 10, thread # = 0
```